

Projeto de Bloco: Engenharia de Softwares Escaláveis [26E2_5]

TP1 — Construção de um Monólito Simples com Spring Boot

TP2 — Camada de Persistência Real com JPA e Spring Data

TP3 — Criação de um Microsserviço com Spring Cloud

Aplicação: Gerenciador de Tarefas (To-Do List)

Arquitetura de Microsserviços — Spring Boot + Spring Cloud + React

Github: [Lucas-1234567890/Projeto-de-Bloco-Engenharia-de-softwares-escal-veis](https://github.com/Lucas-1234567890/Projeto-de-Bloco-Engenharia-de-softwares-escal-veis)

Vídeo Apresentação: https://www.youtube.com/watch?v=h8xdm_LUMzk

Aluno: Lucas Amorim Porciuncula

2026

1. Introdução	4
1.1 Tecnologias Utilizadas	4
2. Domínio e Modelagem DDD	5
2.1 Domínio e Subdomínios	5
2.2 Bounded Context e Linguagem Ubíqua	5
3. Arquitetura da Solução	6
3.1 Fluxo de uma Requisição	6
3.2 Estrutura de Pacotes — Back-end	6
3.3 Estrutura — Front-end	6
4. Back-end com Spring Boot	6
4.1 Entidade Task	6
4.2 Repository	7
4.3 Service	7
4.4 Controller REST	7
4.5 application.properties (TP1)	8
4.6 Endpoints da API	8
5. Front-end com React	9
5.1 Chamadas HTTP (Fetch API)	9
5.2 App.jsx	9
6. Diagramas	9
6.1 Diagrama de Componentes	9
6.2 Diagrama de Sequência — Criar Tarefa	10
6.3 Diagrama de Sequência — Concluir Tarefa	11
7. Instruções de Execução	12
7.1 Pré-requisitos	12
7.2 Banco de Dados (Docker)	12
7.3 Back-end	12
7.4 Front-end	12
8. Princípios SOLID e Clean Code	12
8.1 Single Responsibility (SRP)	12
8.2 Open/Closed (OCP)	13
8.3 Liskov Substitution (LSP)	13
8.4 Interface Segregation (ISP)	13
8.5 Dependency Inversion (DIP)	13
8.6 Clean Code	13
9. Conclusão do TP1	13
10. TP2: Camada de Persistência Real	14
10.1 Motivação da Mudança	14
10.2 Modelagem de Dados — task_history	14
10.2.1 Decisão de modelagem: por que task_id não é uma foreign key física	14
10.2.2 Por que uma tabela própria em vez de Hibernate Envers	15
10.3 Migrations Versionadas (Flyway)	15
10.4 Auditoria Automática com JPA	15

10.5 Repositórios com Paginação e Filtros	16
10.6 Tratamento de Erros	17
10.7 Novo Diagrama de Sequência — Excluir Tarefa (com histórico)	17
10.8 Testes Automatizados	18
10.9 Infraestrutura: Docker Compose	18
11. TP3: Criação de um Microserviço	19
11.1 Motivação da Extração	19
11.2 Atualização do Modelo de Domínio	19
11.3 Arquitetura da Solução — Visão de Microserviços	20
11.4 discovery-service — Servidor Eureka	21
11.5 history-service — Implementação do Microserviço	21
11.6 Repositório do Microserviço	23
11.7 Comunicação entre Serviços e Novos Endpoints REST	24
11.8 Componentes de Front-End	25
11.9 Testes Automatizados	25
11.10 Infraestrutura: Docker Compose com Dois Bancos	26
11.11 Ordem de Inicialização e Descoberta de Serviços	27
12. Conclusão Geral	27

1. Introdução

Este documento descreve a primeira e a segunda entrega do trabalho prático da disciplina. A primeira entrega (TP1) consistiu no desenvolvimento de uma aplicação monolítica utilizando Spring Boot no back-end e React no front-end. A segunda entrega (TP2), detalhada na Seção 10, evolui essa base com uma camada de persistência real utilizando JPA integrado ao Spring Data, incluindo histórico de mudanças e testes automatizados. A terceira entrega (TP3), detalhada na Seção 11, evolui a aplicação para uma arquitetura de microsserviços, extraindo o histórico de auditoria para um serviço independente com descoberta via Eureka.

O domínio escolhido foi um Gerenciador de Tarefas (To-Do List), por ser simples o suficiente para focar nos conceitos arquiteturais exigidos, mas rico o bastante para demonstrar design em camadas com separação de responsabilidades.

A aplicação permite ao usuário criar, listar, atualizar e excluir tarefas, além de marcá-las como concluídas e consultar o histórico de alterações de cada uma.

1.1 Tecnologias Utilizadas

Camada	Tecnologia	Justificativa
Back-end	Spring Boot 3.x	Autoconfig., ecossistema maduro
Banco de Dados (produção)	PostgreSQL 16 (via Docker)	Persistência real; substitui o H2 in-memory do TP1
Banco de Dados (testes)	H2 (in-memory)	Zero configuração, testes rápidos e isolados
Migrations	Flyway	Versionamento de schema, rastreável no Git
Containerização	Docker / Docker Compose	Ambiente reproduzível, elimina divergência entre máquinas
ORM	Spring Data JPA	Abstração do acesso a dados
Front-end	React 18 + Vite	SPA leve, build rápido
Build (back)	Maven	Gerenciamento de dependências
Versionamento	Git / GitHub	Histórico e entrega do código

2. Domínio e Modelagem DDD

Aplicando conceitos de Domain-Driven Design (DDD), foi realizada a análise do domínio antes de qualquer implementação.

2.1 Domínio e Subdomínios

Domínio principal: Gestão de Produtividade Pessoal.

- Core Domain: Gerenciamento de Tarefas — criação, atualização, conclusão e exclusão de tarefas, incluindo rastreabilidade de mudanças.
- Supporting Domain: Categorização — reservado para evoluções futuras (prioridade, tags).
- Generic Domain: Autenticação — não implementado nesta entrega.

2.2 Bounded Context e Linguagem Ubíqua

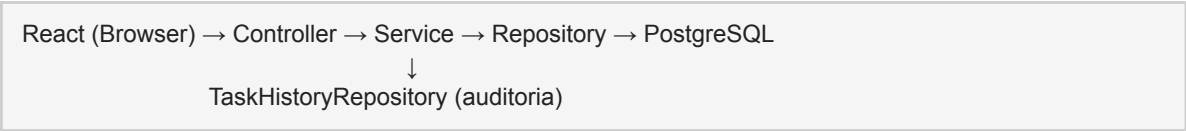
Contexto único nesta entrega: Task Context.

Termo	Significado no Domínio
Tarefa (Task)	Unidade de trabalho a ser realizada pelo usuário
Título	Descrição curta e obrigatória da tarefa
Descrição	Detalhe opcional da tarefa
Concluída (completed)	Estado booleano: tarefa foi finalizada
Concluir Tarefa	Operação que marca completed = true
Atualizar Tarefa	Operação que altera título/descrição de uma tarefa existente
Excluir Tarefa	Remove definitivamente a tarefa do sistema
Histórico (TaskHistory)	Registro imutável de cada evento ocorrido com uma tarefa, para auditoria

3. Arquitetura da Solução

A aplicação segue arquitetura monolítica em camadas (Layered Architecture), com separação clara de responsabilidades entre as camadas de Apresentação, Negócio e Dados.

3.1 Fluxo de uma Requisição



3.2 Estrutura de Pacotes — Back-end

```

todo-api/src/main/java/com/lucas/todo/
├── controller/ TaskController.java      <- recebe requisições HTTP
├── service/    TaskService.java         <- regras de negócio + histórico
├── repository/ TaskRepository.java      <- acesso a Task
├── repository/ TaskHistoryRepository.java <- acesso ao histórico
├── model/      Task.java                <- entidade JPA
├── model/      TaskHistory.java         <- entidade de auditoria
├── model/      TaskAction.java         <- enum de eventos
├── exception/  TaskNotFoundException.java
├── exception/  GlobalExceptionHandler.java
└── TodoApplication.java                <- ponto de entrada

```

3.3 Estrutura — Front-end

```

todo-frontend/src/
├── App.jsx    <- componente raiz, estado + chamadas HTTP
└── main.jsx  <- ponto de entrada React

```

4. Back-end com Spring Boot

4.1 Entidade Task

Classe mapeada para o banco via JPA. Representa o objeto central do domínio (versão do TP1; a versão evoluída com auditoria está na Seção 10.4).

```

@Entity
@Table(name = "tasks")
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;
    private String descricao;
    private boolean completed = false;
    // getters e setters
}

```

4.2 Repository

Estende JpaRepository, herdando automaticamente os métodos de CRUD sem nenhum SQL manual.

```

public interface TaskRepository extends JpaRepository<Task, Long> {
    // findAll, save, findById, deleteById herdados automaticamente
}

```

4.3 Service

Contém toda a lógica de negócio. O Controller nunca acessa o Repository diretamente.

```
@Service
public class TaskService {
    private final TaskRepository repository;
    public TaskService(TaskRepository repository) {
        this.repository = repository;
    }
    public List<Task> listarTodas() { return repository.findAll(); }
    public Task criar(Task task) { return repository.save(task); }
    public Task concluir(Long id) {
        Task task = repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Tarefa não encontrada"));
        task.setCompleted(true);
        return repository.save(task);
    }
    public void deletar(Long id) { repository.deleteById(id); }
}
```

4.4 Controller REST

Expõe os endpoints HTTP e delega toda a lógica ao Service.

```
@RestController
@RequestMapping("/api/tasks")
@CrossOrigin(origins = "**")
public class TaskController {
    private final TaskService service;
    public TaskController(TaskService service) { this.service = service; }
    @GetMapping
    public List<Task> listar() { return service.listarTodas(); }
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Task criar(@RequestBody Task task) { return service.criar(task); }
    @PatchMapping("/{id}/concluir")
    public Task concluir(@PathVariable Long id) { return service.concluir(id); }
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deletar(@PathVariable Long id) { service.deletar(id); }
}
```

4.5 application.properties (TP1)

```
server.port=8080
spring.datasource.url=jdbc:h2:mem:tododb
spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true
spring.jpa.show-sql=true
```

Nota: esta configuração foi substituída na Seção 10 pela configuração de produção com PostgreSQL + Flyway. O H2 permanece em uso, mas restrito ao escopo de testes.

4.6 Endpoints da API

Método	URL	Descrição	Status
GET	/api/tasks?page=0&size=20	Listar tarefas com paginação	200 OK
GET	/api/tasks?completed=true	Filtrar por status	200 OK
GET	/api/tasks?titulo=texto	Buscar por título	200 OK
GET	/api/tasks/{id}	Buscar uma tarefa	200 OK
POST	/api/tasks	Criar nova tarefa	201 Created
PUT	/api/tasks/{id}	Atualizar tarefa	200 OK
PATCH	/api/tasks/{id}/concluir	Marcar como concluída	200 OK
DELETE	/api/tasks/{id}	Excluir tarefa	204 No Content
GET	/api/tasks/{id}/historico	Histórico de eventos da tarefa	200 OK

5. Front-end com React

O front-end foi desenvolvido com React 18 e Vite. Todo o estado e chamadas HTTP estão centralizados no App.jsx, mantendo a estrutura simples.

5.1 Chamadas HTTP (Fetch API)

```
const BASE = 'http://localhost:8080/api/tasks'
// GET - busca todas
fetch(BASE).then(r => r.json()).then(setTasks)
// POST - cria
fetch(BASE, { method: 'POST', headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ titulo, descricao }) })
// PATCH - conclui
fetch(`${BASE}/${id}/concluir`, { method: 'PATCH' })
// DELETE - exclui
fetch(`${BASE}/${id}`, { method: 'DELETE' })
```

5.2 App.jsx

```
export default function App() {
  const [tasks, setTasks] = useState([])
  const [titulo, setTitulo] = useState("")
  useEffect(() => {
    fetch(BASE).then(r => r.json()).then(setTasks)
  }, [])
  function criar(e) {
    e.preventDefault()
    fetch(BASE, { method: 'POST', ... })
      .then(r => r.json())
      .then(nova => setTasks(prev => [...prev, nova]))
  }
  function concluir(id) { /* PATCH -> atualiza estado */ }
  function deletar(id) { /* DELETE -> filtra estado */ }
  return ( <div> <form .../> <ul>{tasks.map(...)}</ul> </div> )
}
```

6. Diagramas

6.1 Diagrama de Componentes

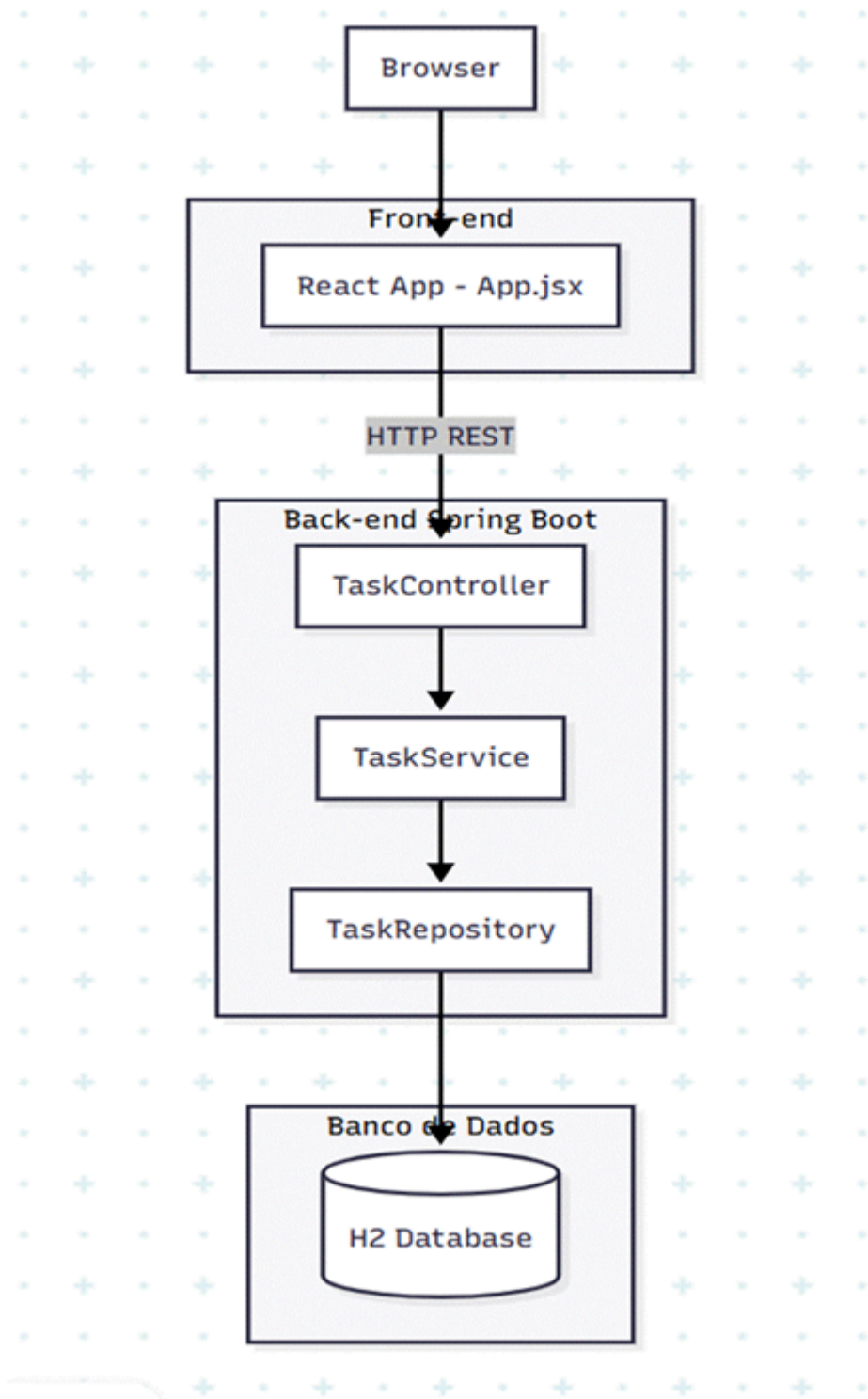


Figura 1 — Diagrama de componentes: Browser → Front-end → Back-end (camadas) → Banco de Dados.

6.2 Diagrama de Sequência — Criar Tarefa

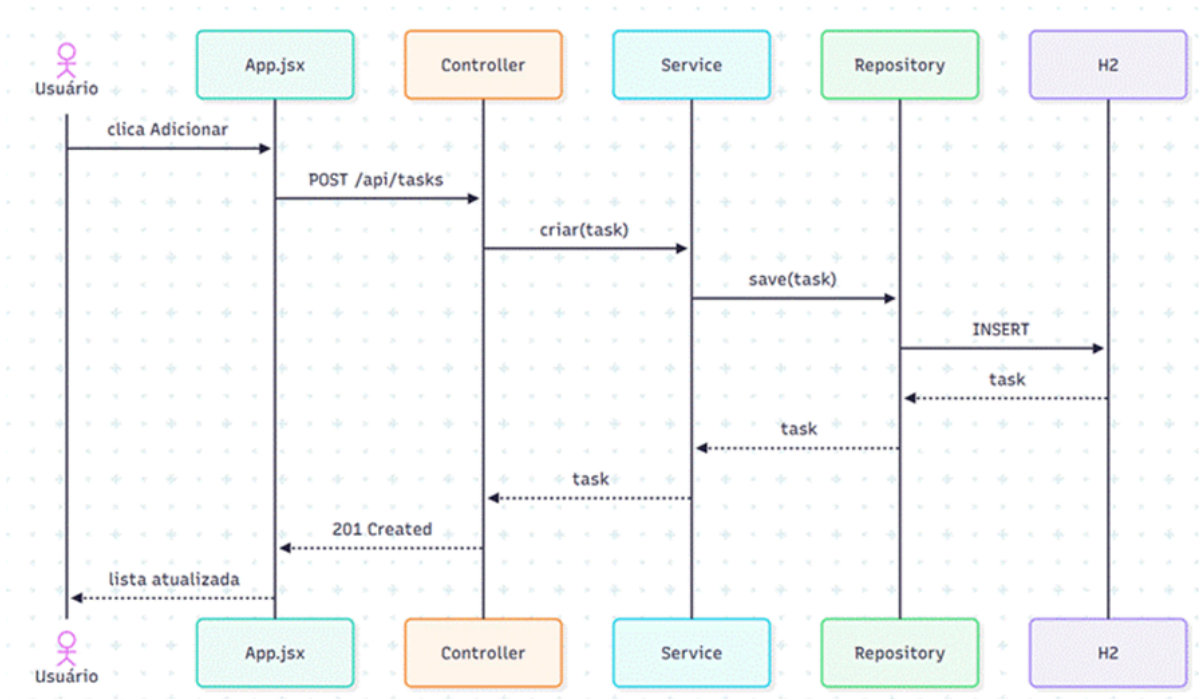


Figura 2 — Fluxo de criação de uma tarefa, do clique do usuário até a resposta 201 Created.

6.3 Diagrama de Sequência — Concluir Tarefa

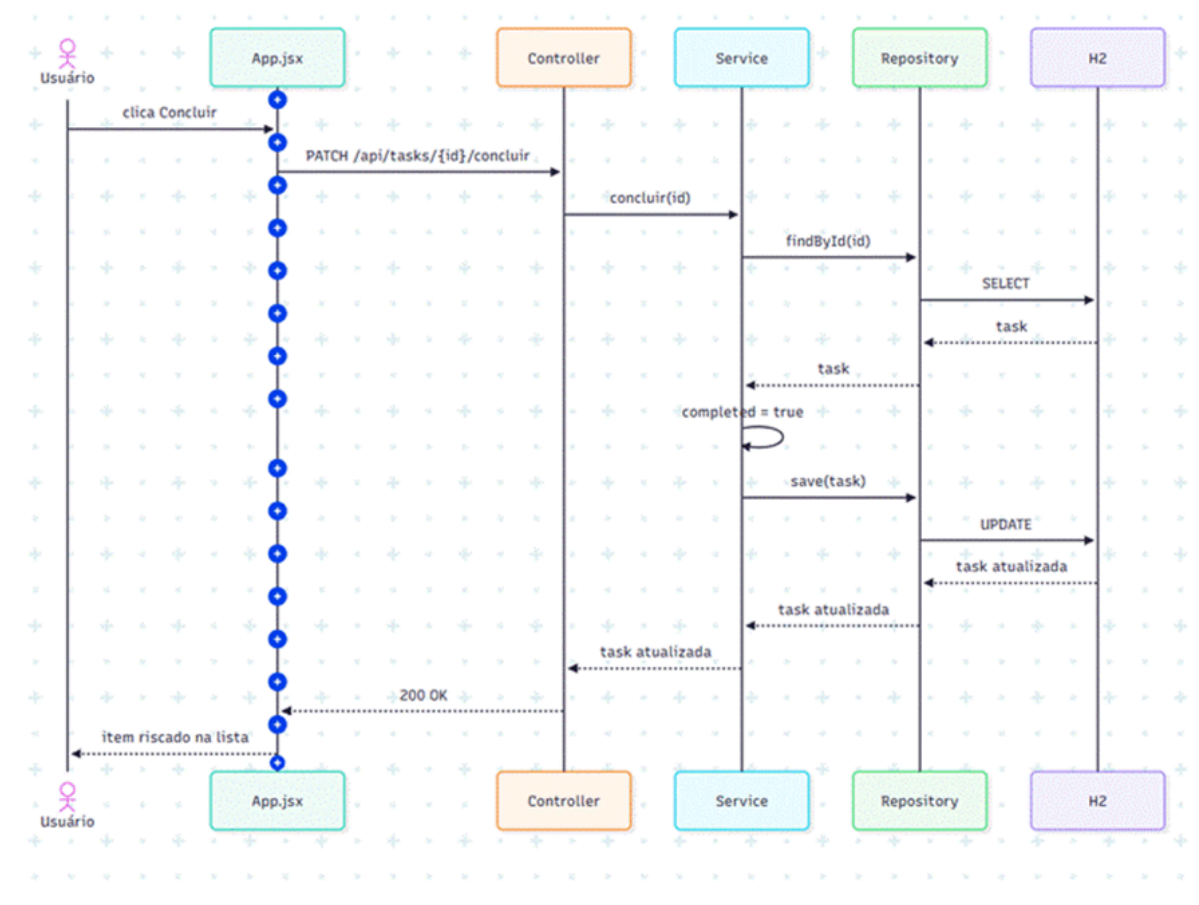


Figura 3 — Fluxo de conclusão de uma tarefa, do clique do usuário até a atualização da lista.

7. Instruções de Execução

7.1 Pré-requisitos

- Java 17 ou superior
- Maven 3.8+
- Node.js 18+ e npm
- Docker e Docker Compose
- Git

7.2 Banco de Dados (Docker)

A partir do TP2, o banco de produção é o PostgreSQL, orquestrado via Docker Compose. Este passo precisa ser executado antes de subir o back-end.

```
docker compose up -d
# Sobe Postgres 16 em localhost:5432
# Banco: tododb | usuário: todo | senha: todo
```

7.3 Back-end

```
cd todo-api
mvn spring-boot:run
# API: http://localhost:8080
# O Flyway aplica as migrations automaticamente na subida
```

Para rodar apenas os testes automatizados (não depende do Docker/Postgres — usa H2 em memória):

```
cd todo-api
mvn test
```

7.4 Front-end

```
cd todo-frontend
npm install
npm run dev
# Interface: http://localhost:5173
```

8. Princípios SOLID e Clean Code

8.1 Single Responsibility (SRP)

Cada classe tem uma única responsabilidade. TaskController cuida apenas do protocolo HTTP; TaskService contém as regras de negócio e a gravação do histórico; TaskRepository e TaskHistoryRepository gerenciam persistência de forma isolada.

8.2 Open/Closed (OCP)

O sistema está aberto para extensão e fechado para modificação. A funcionalidade de histórico (Seção 10) foi adicionada sem alterar os contratos de Controller e Repository já existentes.

8.3 Liskov Substitution (LSP)

O uso de JpaRepository como interface garante que o H2 pode ser substituído por PostgreSQL sem nenhuma alteração no Service ou Controller — o que de fato ocorreu na prática entre o TP1 e o TP2 (ver Seção 10.1).

8.4 Interface Segregation (ISP)

TaskRepository e TaskHistoryRepository expõem apenas os métodos necessários a cada consumidor. Cada função de chamada HTTP no front-end é isolada e cumpre apenas um papel.

8.5 Dependency Inversion (DIP)

TaskService depende das abstrações TaskRepository e TaskHistoryRepository (interfaces JPA), não de implementações concretas. Injeção via construtor em ambos os casos.

8.6 Clean Code

- Nomes descritivos: listarTodas, concluir, deletar, historico.
- Métodos pequenos: cada método do Service faz exatamente uma coisa.
- Tratamento de erro explícito e estruturado via GlobalExceptionHandler (Seção 10).

9. Conclusão do TP1

Esta entrega apresentou a implementação de uma aplicação monolítica completa seguindo boas práticas de engenharia de software. Os objetivos foram atendidos:

- Arquitetura em camadas: Controller → Service → Repository.
- Autoconfiguração do Spring Boot — sem XML de configuração manual.
- API REST funcional com GET, POST, PATCH e DELETE.
- Princípios SOLID aplicados em todas as camadas.
- Modelagem DDD com domínio, subdomínios e linguagem ubíqua definidos.
- Front-end React consumindo a API, com gerenciamento de estado simples.

A base monolítica construída foi projetada para evoluir: o H2 poderia ser substituído por PostgreSQL sem impacto nas camadas superiores, e o Service poderia ser extraído como microserviço em entregas futuras. A Seção 10 confirma essa previsão na prática.

10. TP2: Camada de Persistência Real

A segunda etapa do projeto integrado teve como objetivo evoluir a persistência da aplicação de um banco in-memory descartável para uma camada robusta, com JPA integrado ao Spring Data, histórico de mudanças e testes automatizados — cobrindo as subcompetências de modelagem de dados, integração JPA/Spring Data, gerenciamento de dados, histórico e testes.

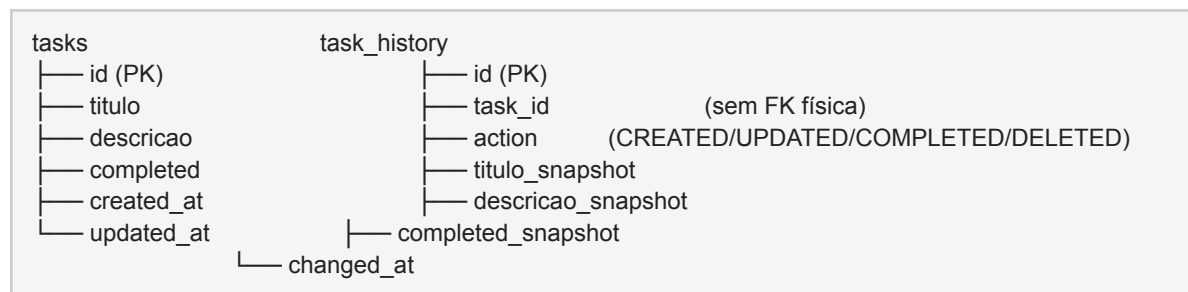
10.1 Motivação da Mudança

No TP1, o banco de dados era H2 in-memory com `spring.jpa.hibernate.ddl-auto=create-drop`: o schema era recriado e todos os dados perdidos a cada reinicialização da aplicação. Essa configuração é adequada para prototipagem, mas inviabiliza qualquer cenário real de auditoria ou histórico — não há como rastrear mudanças em dados que não sobrevivem a um restart.

A substituição por PostgreSQL confirma, na prática, a previsão feita na conclusão do TP1 (Seção 9) e o princípio de Liskov Substitution descrito na Seção 8.3: a troca de banco não exigiu nenhuma alteração em Service ou Controller, apenas na configuração de acesso a dados e no driver JDBC.

10.2 Modelagem de Dados — task_history

Para atender ao requisito de histórico, foi criada uma segunda tabela, `task_history`, que registra um snapshot imutável do estado da tarefa a cada evento relevante (criação, atualização, conclusão ou exclusão).



10.2.1 Decisão de modelagem: por que task_id não é uma foreign key física

O histórico existe justamente para sobreviver à exclusão da task original — é o registro de auditoria de que ela existiu e do que aconteceu com ela ao longo do tempo. Se `task_id` fosse uma FK com `ON DELETE CASCADE`, o histórico seria apagado junto com a task,

anulando o propósito da funcionalidade. Se fosse uma FK sem cascade, a própria operação de exclusão da task quebraria por violação de integridade referencial. Por isso, esse relacionamento é tratado como lógico, resolvido na camada de aplicação, e não imposto pelo schema do banco.

10.2.2 Por que uma tabela própria em vez de Hibernate Envers

Hibernate Envers automatiza a auditoria de entidades, mas esconde o "como": é uma solução opaca, difícil de customizar quando é necessário cruzar eventos de histórico com métricas de negócio específicas (por exemplo, tempo médio até conclusão de uma tarefa, ou taxa de reabertura). Uma tabela explícita com snapshot dá controle total sobre o que é gravado e como é consultado, e evolui naturalmente para relatórios de auditoria — trade-off deliberado entre automatismo e controle.

10.3 Migrations Versionadas (Flyway)

O schema do banco deixou de ser gerado automaticamente pelo Hibernate (ddl-auto=create-drop) e passou a ser controlado por scripts SQL versionados no Flyway, aplicados automaticamente na inicialização da aplicação.

```
-- V1__create_tasks_table.sql
CREATE TABLE tasks (
    id BIGSERIAL PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    descricao VARCHAR(1000),
    completed BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT now(),
    updated_at TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_tasks_completed ON tasks (completed);

-- V2__create_task_history_table.sql
CREATE TABLE task_history (
    id BIGSERIAL PRIMARY KEY,
    task_id BIGINT NOT NULL,
    action VARCHAR(20) NOT NULL,
    titulo_snapshot VARCHAR(200),
    descricao_snapshot VARCHAR(1000),
    completed_snapshot BOOLEAN,
    changed_at TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_task_history_task_id ON task_history (task_id);
CREATE INDEX idx_task_history_changed_at ON task_history (changed_at);
```

Em produção, `spring.jpa.hibernate.ddl-auto=validate`: o Hibernate apenas confere se o schema mapeado bate com o schema real, mas quem cria e altera tabelas é exclusivamente o Flyway. Isso torna toda mudança de schema rastreável e revisável no Git, ao contrário de um schema gerado implicitamente.

10.4 Auditoria Automática com JPA

A entidade Task foi estendida com os campos createdAt e updatedAt, preenchidos automaticamente pelo próprio JPA através de @CreatedDate e @LastModifiedDate, habilitados por @EnableJpaAuditing na classe principal.

```
@Entity
@Table(name = "tasks")
@EntityListeners(AuditingEntityListener.class)
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Título é obrigatório")
    @Column(nullable = false, length = 200)
    private String titulo;

    @Column(nullable = false)
    private boolean completed = false;

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;
    // getters e setters
}
```

Já o histórico de eventos (TaskHistory) é gravado explicitamente no TaskService, e não por meio de um listener de ciclo de vida do JPA (@PostPersist/@PostUpdate). Essa escolha mantém a lógica de auditoria visível e testável, sem depender de mecanismos implícitos de callback:

```
@Transactional
public Task concluir(Long id) {
    Task task = buscarPorId(id);
    task.setCompleted(true);
    Task concluida = repository.save(task);
    registrarHistorico(concluida, TaskAction.COMPLETED);
    return concluida;
}

@Transactional
public void deletar(Long id) {
    Task task = buscarPorId(id);
    registrarHistorico(task, TaskAction.DELETED); // snapshot antes de apagar
    repository.delete(task);
}
```

10.5 Repositórios com Paginação e Filtros

O TaskRepository do TP1 herdava apenas os métodos genéricos de JpaRepository (findAll, save, findById, deleteById). No TP2, foram adicionados métodos derivados de nome de query do Spring Data, com suporte nativo a paginação via Pageable:

```

public interface TaskRepository extends JpaRepository<Task, Long> {
    Page<Task> findByCompleted(boolean completed, Pageable pageable);
    Page<Task> findByTituloContainingIgnoreCase(String titulo, Pageable pageable);
}

public interface TaskHistoryRepository extends JpaRepository<TaskHistory, Long> {
    List<TaskHistory> findByTaskIdOrderByChangedAtDesc(Long taskId);
}

```

Exemplos de uso:

```

// Paginação + filtro por status
Page<Task> pendentes = taskRepository.findByCompleted(false, PageRequest.of(0, 20));

// Busca por título, case-insensitive
Page<Task> resultado = taskRepository.findByTituloContainingIgnoreCase("relatório", pageable);

// Histórico completo de uma tarefa, mais recente primeiro
List<TaskHistory> eventos = taskHistoryRepository.findByTaskIdOrderByChangedAtDesc(taskId);

```

Sem paginação, a listagem completa (GET /api/tasks) funcionaria com poucas dezenas de registros em desenvolvimento, mas degradaria em produção conforme a base de tarefas crescesse — um ponto de atenção já identificado na revisão do TP1.

10.6 Tratamento de Erros

O TP1 lançava `RuntimeException` genérica quando uma tarefa não era encontrada, resultando em resposta HTTP 500 para o cliente. No TP2, essa lacuna foi fechada com uma exceção de domínio e um handler global:

```

public class TaskNotFoundException extends RuntimeException {
    public TaskNotFoundException(Long id) {
        super("Tarefa não encontrada: id=" + id);
    }
}

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(TaskNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(TaskNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(...);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(...) { ... }
}

```

10.7 Novo Diagrama de Sequência — Excluir Tarefa (com histórico)

O diagrama abaixo ilustra a diferença mais importante introduzida pela camada de persistência do TP2 em relação ao TP1: antes de qualquer exclusão física, o Service grava o snapshot do estado atual da tarefa em `task_history`.

Figura 4 — Fluxo de exclusão de uma tarefa: o histórico é gravado antes da exclusão física do registro.

10.8 Testes Automatizados

Foram implementados testes cobrindo as três camadas envolvidas na persistência, cada um com a ferramenta mais adequada ao seu escopo:

Camada	Tipo de Teste	Ferramenta
TaskRepository	Integração com banco real (H2)	@DataJpaTest
TaskService	Unitário, com mocks dos repositórios	JUnit 5 + Mockito
TaskController	Integração HTTP simulada	@WebMvcTest + MockMvc

Os testes cobrem: paginação e filtros de listagem; geração correta de um evento de histórico para cada ação (criar, atualizar, concluir, deletar); tratamento de 404 para tarefas inexistentes; e validação de entrada (400 quando o título está em branco). Os testes de integração usam H2 em memória, configurado exclusivamente em `src/test/resources/application.properties`, e não dependem do Postgres nem do Docker — podem rodar em qualquer ambiente, inclusive em pipelines de CI.

```
@Test
void concluirDeveMarcarCompletedERegistrarHistoricoDeConclusao() {
    Task existente = new Task("Tarefa", null);
    existente.setId(5L);
    when(repository.findById(5L)).thenReturn(Optional.of(existente));
    when(repository.save(any(Task.class))).thenAnswer(inv -> inv.getArgument(0));

    Task resultado = service.concluir(5L);

    assertThat(resultado.isCompleted()).isTrue();
    ArgumentCaptor<TaskHistory> captor = ArgumentCaptor.forClass(TaskHistory.class);
    verify(historyRepository).save(captor.capture());
    assertThat(captor.getValue().getAction()).isEqualTo(TaskAction.COMPLETED);
}
```

10.9 Infraestrutura: Docker Compose

O PostgreSQL de desenvolvimento é provisionado via Docker Compose, escolha alinhada ao restante do fluxo de trabalho do autor (que já utiliza Docker para orquestrar stacks de automação em outros projetos). Isso evita conflito com instalações locais de banco já existentes na máquina e garante que qualquer pessoa que clone o repositório suba o mesmo ambiente com um único comando.

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: tododb
      POSTGRES_USER: todo
      POSTGRES_PASSWORD: todo
    ports:
      - "5432:5432"
    volumes:
      - todo_pg_data:/var/lib/postgresql/data
```

O uso do Docker não é uma exigência técnica da camada de persistência em si — o mesmo schema Flyway funcionaria contra qualquer instância Postgres, local ou gerenciada na nuvem. É uma decisão de conveniência de ambiente, mantendo o princípio de Liskov Substitution também no nível de infraestrutura: trocar a origem do Postgres não exige nenhuma mudança de código.

11. TP3: Criação de um Microserviço

A terceira entrega evolui a aplicação de um monólito em camadas (TP1) com persistência real (TP2) para uma arquitetura de microserviços. A funcionalidade de histórico/auditoria de tarefas, que no TP2 vivia dentro do próprio todo-api, foi extraída para um serviço independente — o history-service — com banco de dados próprio, descoberta dinâmica via Eureka e comunicação declarativa via Feign. Esta seção cobre as subcompetências exigidas pela etapa: atualização do modelo de domínio, novos endpoints REST, implementação do microserviço com Spring Boot e Spring Cloud, repositório dedicado, componentes de front-end e expansão da cobertura de testes.

11.1 Motivação da Extração

No TP2, o histórico já era tratado como uma responsabilidade auxiliar dentro do todo-api: gravado pelo TaskService, mas fisicamente hospedado no mesmo banco tododb. Extrair essa responsabilidade para um serviço próprio torna essa fronteira explícita em vez de apenas lógica: auditoria passa a poder evoluir, escalar e falhar de forma independente do CRUD principal de tarefas — sem exigir deploy conjunto e sem compartilhar schema de banco. É a mesma divisão de responsabilidade identificada na conclusão do TP2 (Seção 11) como evolução natural, agora implementada.

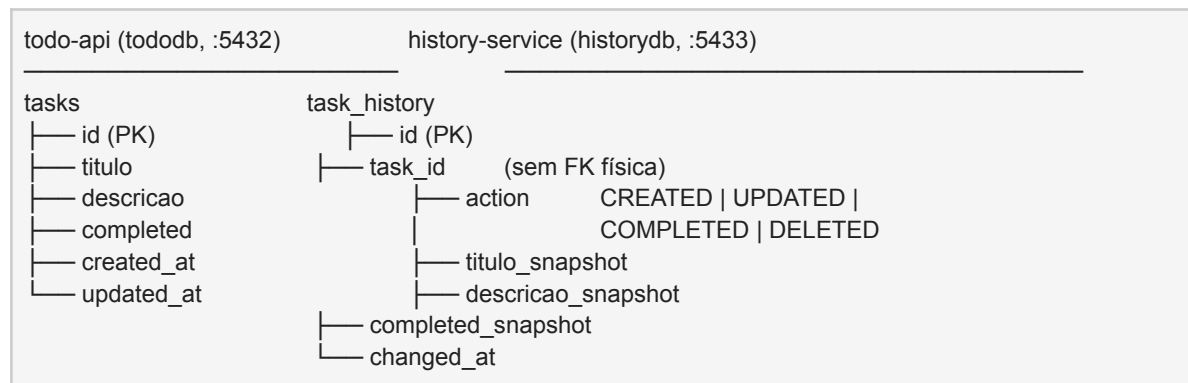
11.2 Atualização do Modelo de Domínio

O Bounded Context único do TP1/TP2 (Task Context) foi dividido em dois contextos delimitados, cada um com seu próprio modelo e ciclo de vida:

- Task Context (todo-api) — permanece dono da entidade Task e das regras de CRUD.

- History Context (history-service) — passa a ser o único dono de TaskHistory, incluindo a tabela task_history e a lógica de cálculo de estatísticas (tempo até conclusão, reaberturas).

A comunicação entre os dois contextos deixa de ser uma chamada de método Java dentro do mesmo processo e passa a ser um contrato HTTP explícito, com DTOs próprios em cada lado (TaskHistoryRequest, TaskHistoryResponse, TaskStatsResponse duplicados em todo-api e history-service) — nenhuma classe de domínio é compartilhada via biblioteca comum, evitando acoplamento indevido entre os serviços.

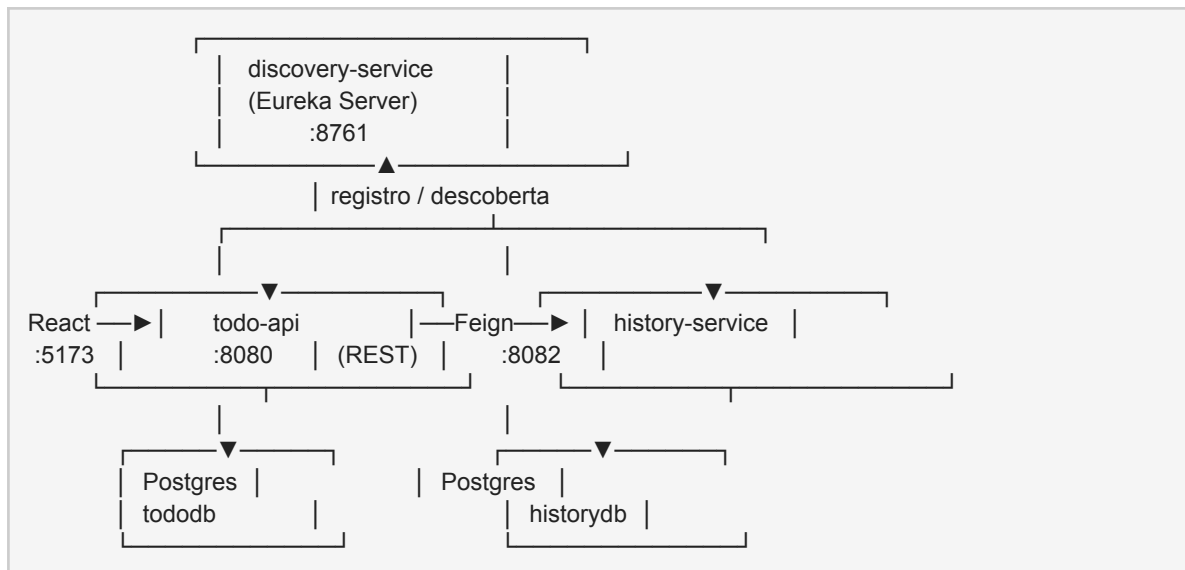


A migration V3__drop_task_history_table.sql, adicionada ao todo-api, documenta essa migração de propriedade no próprio versionamento do schema: a tabela task_history é removida do tododb porque seus dados passam a residir exclusivamente no historydb.

```
-- V3__drop_task_history_table.sql
-- A tabela task_history foi extraída para um microsserviço dedicado
-- (history-service), com banco de dados próprio. O todo-api deixa de ser
-- dono desses dados; a partir de agora ele fala com o history-service
-- via REST (Feign) em vez de acessar a tabela diretamente.
DROP TABLE IF EXISTS task_history;
```

11.3 Arquitetura da Solução — Visão de Microsserviços

Um terceiro serviço, o discovery-service, foi introduzido exclusivamente para descoberta: nenhum dos outros dois serviços conhece o host/porta um do outro em tempo de compilação — cada um se registra no Eureka ao subir e consulta o registro quando precisa localizar outro serviço.



O front-end continua conhecendo apenas o todo-api — a existência do history-service é um detalhe de infraestrutura interno, invisível para o React.

11.4 discovery-service — Servidor Eureka

O discovery-service é um projeto Spring Boot dedicado exclusivamente ao papel de servidor de registro, habilitado por `@EnableEurekaServer`. Ele não se registra em si mesmo nem consulta outros serviços — apenas mantém o registro de quem está disponível e em qual endereço.

```

@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServiceApplication.class, args);
    }
}
  
```

```

spring.application.name=discovery-service
server.port=8761
  
```

```

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
eureka.instance.hostname=localhost
  
```

A dependência chave no pom.xml é `spring-cloud-starter-netflix-eureka-server`, gerenciada pelo BOM `spring-cloud-dependencies` (versão 2023.0.3), o mesmo padrão do Spring Cloud usado em `todo-api` e `history-service` para garantir compatibilidade de versões entre os três módulos.

11.5 history-service — Implementação do Microserviço

O history-service é um segundo projeto Spring Boot independente, com seu próprio pom.xml, seu próprio banco PostgreSQL (`historydb`, porta 5433) e suas próprias migrations

Flyway — reaproveitando a mesma tabela `task_history` criada no TP2, agora dona exclusiva desse serviço.

Dependências principais adicionadas ao `pom.xml`:

- `spring-cloud-starter-netflix-eureka-client` — registra o serviço no Eureka como "history-service" ao subir.
- `spring-boot-starter-data-jpa + postgresql` — persistência própria.
- `flyway-core + flyway-database-postgresql` — migrations versionadas, mesmo padrão do `todo-api`.
- `spring-boot-starter-validation` — validação dos DTOs de entrada.

```
spring.application.name=history-service
server.port=8082

spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5433/historydb}
spring.datasource.username=${DB_USER:history}
spring.datasource.password=${DB_PASSWORD:history}

spring.jpa.hibernate.ddl-auto=validate
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration

eureka.client.service-url.defaultZone=${EUREKA_URI:http://localhost:8761/eureka}
eureka.instance.prefer-ip-address=true
```

A entidade `TaskHistory` foi migrada do `todo-api` sem alterações no schema — apenas trocou de módulo/pacote (com.lucas.history) — reforçando que a extração é uma mudança de fronteira arquitetural, não de modelagem de dados:

```
@Entity
@Table(name = "task_history")
public class TaskHistory {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "task_id", nullable = false)
    private Long taskId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private TaskAction action;

    private String tituloSnapshot;
    private String descricaoSnapshot;
    private Boolean completedSnapshot;
    private LocalDateTime changedAt;
    // getters e setters
}
```

A camada de serviço (`TaskHistoryService`) ganhou uma responsabilidade nova em relação ao TP2: o cálculo de estatísticas derivadas do histórico — tempo até a primeira conclusão e número de reaberturas — que antes não existia:

```

public TaskStatsResponse estatisticas(Long taskId) {
    List<TaskHistory> eventos =
        repository.findByTaskIdOrderByChangedAtAsc(taskId);

    LocalDateTime criadoEm = eventos.stream()
        .filter(e -> e.getAction() == TaskAction.CREATED)
        .map(TaskHistory::getChangedAt).findFirst().orElse(null);

    List<LocalDateTime> conclusoes = eventos.stream()
        .filter(e -> e.getAction() == TaskAction.COMPLETED)
        .map(TaskHistory::getChangedAt).toList();

    LocalDateTime primeiraConclusao =
        conclusoes.isEmpty() ? null : conclusoes.get(0);

    // Reabertura = task concluída mais de uma vez
    // (concluir -> atualizar -> concluir de novo)
    int reaberturas = Math.max(0, conclusoes.size() - 1);
    ...
}

```

11.6 Repositório do Microserviço

O TaskHistoryRepository do history-service é um repositório dedicado, isolado do TaskRepository do todo-api — cada serviço só enxerga as entidades do seu próprio contexto:

```

public interface TaskHistoryRepository
    extends JpaRepository<TaskHistory, Long> {

    List<TaskHistory> findByTaskIdOrderByChangedAtDesc(Long taskId);

    List<TaskHistory> findByTaskIdOrderByChangedAtAsc(Long taskId);
}

```

A migration Flyway V1__create_task_history_table.sql do history-service recria o schema de forma independente do todo-api, incluindo os mesmos índices de consulta por task e por data já validados no TP2:

```

CREATE TABLE task_history (
    id      BIGSERIAL PRIMARY KEY,
    task_id BIGINT NOT NULL,
    action  VARCHAR(20) NOT NULL,
    titulo_snapshot VARCHAR(200),
    descricao_snapshot VARCHAR(1000),
    completed_snapshot BOOLEAN,
    changed_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE INDEX idx_task_history_task_id ON task_history (task_id);
CREATE INDEX idx_task_history_changed_at ON task_history (changed_at);

```

11.7 Comunicação entre Serviços e Novos Endpoints REST

O todo-api consome o history-service através de um Feign Client declarativo. O atributo name do @FeignClient é o mesmo spring.application.name registrado no Eureka pelo history-service — nenhuma URL é hardcoded no código, o endereço é resolvido em tempo de execução:

```
@FeignClient(name = "history-service")
public interface HistoryClient {

    @PostMapping("/api/history")
    TaskHistoryResponse registrarEvento(@RequestBody TaskHistoryRequest request);

    @GetMapping("/api/history/task/{taskId}")
    List<TaskHistoryResponse> buscarHistorico(@PathVariable("taskId") Long taskId);

    @GetMapping("/api/history/task/{taskId}/estatisticas")
    TaskStatsResponse buscarEstatisticas(@PathVariable("taskId") Long taskId);
}
```

Resiliência: no TaskService, o registro de eventos é envolto em try/catch de FeignException. Uma falha de comunicação com o history-service é logada como warning, mas não impede a operação principal de CRUD na task — auditoria é uma funcionalidade auxiliar, não deve travar o fluxo principal:

```
private void registrarHistorico(Task task, TaskAction action) {
    try {
        TaskHistoryRequest request = new TaskHistoryRequest(
            task.getId(), action.name(), task.getTitulo(),
            task.getDescricao(), task.isCompleted());
        historyClient.registrarEvento(request);
    } catch (FeignException ex) {
        log.warn("Falha ao registrar histórico da task {} (ação={}): {}",
            task.getId(), action, ex.getMessage());
    }
}
```

Endpoints expostos pelo todo-api ao front-end (inalterados na assinatura, agora atuando como fachada para o history-service):

Método	Endpoint	Descrição
GET	/api/tasks/{id}/historico	Histórico da tarefa (proxy para history-service)
GET	/api/tasks/{id}/estatisticas	Estatísticas da tarefa (novo — proxy para history-service)

Endpoints internos do history-service, consumidos apenas pelo todo-api via Feign:

Método	Endpoint	Descrição
--------	----------	-----------

POST	/api/history	Registrar um evento de auditoria
GET	/api/history/task/{taskId}	Listar eventos de uma tarefa
GET	/api/history/task/{taskId}/estatisticas	Métricas derivadas do histórico

O tratamento de erros também atravessa a fronteira dos serviços: o TaskService traduz um FeignException.NotFound vindo do history-service em uma TaskNotFoundException local, mantendo o mesmo contrato de erro 400/404 já usado no TP2, mesmo quando o dado vem de outro processo.

11.8 Componentes de Front-End

Foi adicionado o componente HistoryModal.jsx, que consome os novos endpoints /historico e /estatisticas do todo-api e exibe, em um modal, a linha do tempo de eventos da tarefa e um painel de métricas (total de eventos, conclusões, reaberturas e tempo até a primeira conclusão). O App.jsx ganhou um botão "🕒 Histórico" por tarefa, que abre o modal passando o id da tarefa selecionada:

```
{taskSelecionada && (
  <HistoryModal
    taskId={taskSelecionada.id}
    titulo={taskSelecionada.titulo}
    onClose={() => setTaskSelecionada(null)}
  />
)}

useEffect(() => {
  Promise.all([
    fetch(`${BASE}/${taskId}/historico`)
      .then(r => (r.ok ? r.json() : Promise.reject(r.status))),
    fetch(`${BASE}/${taskId}/estatisticas`)
      .then(r => (r.ok ? r.json() : Promise.reject(r.status))),
  ])
  .then(([historico, estatisticas]) => {
    setEventos(historico)
    setStats(estatisticas)
  })
  .catch(() => setErro('Não foi possível carregar o histórico (history-service indisponível?).'))
}, [taskId])
```

O front-end continua fazendo fetch apenas contra http://localhost:8080 — nenhuma alteração de host foi necessária, reforçando que a extração do microserviço é transparente para a camada de apresentação.

11.9 Testes Automatizados

A cobertura de testes foi replicada no history-service, seguindo o mesmo padrão de três camadas já estabelecido no TP2 para o todo-api:

Camada	Tipo de Teste	Ferramenta
--------	---------------	------------

TaskHistoryRepository	Integração com banco real (H2)	@DataJpaTest
TaskHistoryService	Unitário, com mocks do repositório	JUnit 5 + Mockito
TaskHistoryController	Integração HTTP simulada	@WebMvcTest + MockMvc

Os testes cobrem: gravação de eventos e conversão para DTO de resposta; consulta de histórico ordenado por data; cálculo de estatísticas (tempo até conclusão, contagem de reaberturas); 404 via `HistoricoNaoEncontradoException` quando a task não possui nenhum evento; e validação de entrada (400 quando `taskId` ou `action` estão ausentes). Assim como no `todo-api`, os testes usam H2 em memória, isolados do PostgreSQL de produção, e podem rodar em qualquer ambiente, inclusive em pipelines de CI.

```
@Test
void estatisticasDeveCalcularTempoAteConclusaoEReaberturas() {
    // arrange: eventos CREATED -> COMPLETED -> UPDATED -> COMPLETED
    when(repository.findByTaskIdOrderByChangedAtAsc(5L))
        .thenReturn(eventosDeExemplo);

    TaskStatsResponse stats = service.estatisticas(5L);

    assertThat(stats.getQuantidadeConclusoes()).isEqualTo(2);
    assertThat(stats.getReaberturas()).isEqualTo(1);
    assertThat(stats.getTempoAteConclusaoSegundos()).isNotNull();
}
```

11.10 Infraestrutura: Docker Compose com Dois Bancos

O `docker-compose.yml` foi expandido para orquestrar dois bancos Postgres isolados — um por serviço, seguindo o padrão `database-per-service` — em vez de compartilhar uma única instância:

```

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: tododb
      POSTGRES_USER: todo
      POSTGRES_PASSWORD: todo
    ports:
      - "5432:5432"

# Banco próprio do history-service (database-per-service): cada
# microserviço tem seu schema isolado, sem acesso direto de um
# serviço ao banco do outro.
history-db:
  image: postgres:16-alpine
  environment:
    POSTGRES_DB: historydb
    POSTGRES_USER: history
    POSTGRES_PASSWORD: history
  ports:
    - "5433:5432"

```

11.11 Ordem de Inicialização e Descoberta de Serviços

Diferente do TP1/TP2, a ordem de subida dos processos passa a importar: o discovery-service precisa estar disponível antes que todo-api e history-service consigam se registrar, e o history-service precisa estar registrado antes que o todo-api consiga resolvê-lo via Feign na primeira chamada.

```

# 1. Bancos
docker compose up -d

# 2. Eureka primeiro (os outros dependem dele pra registrar)
cd discovery-service && mvn spring-boot:run
# aguarda subir em http://localhost:8761

# 3. history-service (registra no Eureka como "history-service")
cd history-service && mvn spring-boot:run

# 4. todo-api (registra como "todo-api" e descobre o
#   history-service via Eureka)
cd todo-api && mvn spring-boot:run

# 5. Frontend
cd todo-frontend && npm run dev

```

O dashboard do Eureka em <http://localhost:8761> permite acompanhar visualmente os dois serviços se registrando (INSTANCES CURRENTLY REGISTERED WITH EUREKA), útil tanto para depuração local quanto para a demonstração da entrega.

12. Conclusão Geral

O TP3 evoluiu a aplicação de uma persistência real e auditável (TP2) para uma arquitetura distribuída funcional. Os objetivos da etapa foram atendidos:

- Modelo de domínio atualizado, com o History Context isolado em um serviço e banco próprios (database-per-service).
- Novos endpoints REST (/estatisticas) e comunicação entre serviços via Feign, com descoberta dinâmica via Eureka.
- Microserviço implementado com Spring Boot e Spring Cloud (Eureka Client/Server, OpenFeign), com Flyway e testes independentes.
- Repositório dedicado (TaskHistoryRepository) isolado do restante do sistema, sem FK física entre bancos de serviços diferentes.
- Componente de front-end (HistoryModal) consumindo os dados do microserviço através de um único host conhecido pela interface.
- Cobertura de testes expandida para as três camadas do history-service, espelhando o padrão já validado no todo-api.

A extração confirma, na prática, um princípio já discutido nas conclusões do TP1 e do TP2: por depender de abstrações (JpaRepository, interfaces Feign) e não de implementações concretas, o Service não precisou ser reescrito para passar a falar com outro processo — apenas a fonte dos dados de histórico mudou de uma chamada de repositório local para uma chamada HTTP resolvida via Eureka. Para as próximas etapas, os pontos identificados como evolução natural são: um API Gateway único na frente dos serviços, circuit breaker (Resilience4j) nas chamadas Feign, e autenticação entre serviços.

Lucas Amorim Porciuncula